



PRÉSENTATION TECHNIQUE

La portée des variables en PHP

Où vit une variable ? Qui peut y accéder ?

Guide simple et détaillé — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Qu'est-ce que la portée d'une variable ?

DÉFINITION

La **portée** (en anglais *scope*) d'une variable désigne **la zone du code dans laquelle cette variable existe et est accessible**. En dehors de cette zone, la variable n'existe tout simplement pas : PHP ne peut ni la lire ni la modifier.

ANALOGIE SIMPLE

Imagine une école avec plusieurs salles de classe. Une règle posée dans la salle A n'existe que dans la salle A : les élèves de la salle B ne peuvent pas la voir ni l'utiliser. En PHP, c'est pareil : une variable créée dans une fonction ne "vit" que dans cette fonction. Elle n'existe pas en dehors.

Les trois niveaux de portée en PHP

Portée	Où la variable est créée	Où elle est accessible
Locale	À l'intérieur d'une fonction	Uniquement dans cette fonction
Globale	En dehors de toute fonction	Partout hors des fonctions (sauf si on le demande explicitement)
Statique	À l'intérieur d'une fonction, avec <code>static</code>	Dans la fonction, mais sa valeur est mémorisée entre les appels

```
ZONE GLOBALE (en dehors des fonctions)
$nom = "Khadija"; ← variable globale
$age = 19; ← variable globale
```

```
| ZONE LOCALE (à l'intérieur d'une fonction) |
| $message = "Bonjour"; ← variable locale |
| $message n'existe PAS en dehors d'ici |
```

2. La variable locale

DÉFINITION

Une variable est dite **locale** quand elle est créée à l'intérieur d'une fonction. Elle n'existe que pendant l'exécution de cette fonction, et disparaît dès que la fonction se termine. Elle est complètement invisible depuis l'extérieur.

Exemple : une variable locale est invisible depuis l'extérieur

```
<?php
function direBonjour() {
    $message = "Bonjour !";    // variable LOCALE
    echo $message;             // fonctionne ici
}

direBonjour();    // Affiche : Bonjour !

echo $message;    // ERREUR : $message n'existe pas ici
?>
```

CE QUI SE PASSE EXACTEMENT

Quand PHP appelle `direBonjour()`, il crée la variable `$message` en mémoire. Quand la fonction se termine, PHP efface cette variable de la mémoire. Elle n'existe plus nulle part. Essayer d'y accéder depuis l'extérieur retourne une valeur vide (et une notice d'avertissement en mode débogage).

Deux fonctions différentes : leurs variables locales sont séparées

Même si deux fonctions utilisent une variable avec le même nom, elles sont totalement indépendantes : chacune a sa propre "boîte" pour ce nom.

```
<?php
function fonctionA() {
    $score = 10;
    echo "A : " . $score;    // Affiche : A : 10
}

function fonctionB() {
    $score = 99;
    echo "B : " . $score;    // Affiche : B : 99
}

fonctionA();    // A : 10
fonctionB();    // B : 99
// Les deux $score sont indépendants, aucun conflit
?>
```

POURQUOI C'EST UTILE

Cette isolation est une protection : si deux fonctions utilisent une variable nommée `$total`, elles ne se perturbent jamais l'une l'autre. Chacune travaille dans sa propre zone privée, ce qui rend le code plus sûr et plus prévisible.

3. La variable globale

DÉFINITION

Une variable est dite **globale** quand elle est créée en dehors de toute fonction. Elle est accessible partout dans le fichier PHP, sauf à l'intérieur des fonctions — à moins d'utiliser explicitement le mot-clé `global`.

Une variable globale n'est PAS automatiquement visible dans une fonction

C'est le piège le plus fréquent. En PHP, une fonction ne voit pas automatiquement les variables créées à l'extérieur.

```
<?php
$prenom = "Khadija";    // variable GLOBALE

function afficherPrenom() {
    echo $prenom;        // Ne fonctionnera PAS comme prévu
                        // PHP cherche $prenom EN LOCAL et ne la trouve pas
                        // Affiche : (rien / avertissement)
}

afficherPrenom();

?>
```

Solution : déclarer la variable avec le mot-clé "global"

Pour utiliser une variable globale à l'intérieur d'une fonction, il faut la déclarer avec le mot-clé `global` au début de la fonction. PHP comprend alors qu'il doit aller chercher cette variable dans la zone globale.

```
<?php
$prenom = "Khadija";    // variable GLOBALE

function afficherPrenom() {
    global $prenom;      // on dit à PHP : utilise la variable globale
    echo $prenom;        // Affiche : Khadija
}

afficherPrenom();

?>
```

On peut aussi modifier une variable globale depuis une fonction

```
<?php
$compteur = 0;

function incrementer() {
    global $compteur;
    $compteur++;         // modifie directement la variable globale
}

incrementer();
incrementer();
incrementer();

echo $compteur;         // Affiche : 3

?>
```

CONSEIL IMPORTANT

Utiliser `global` fonctionne, mais en excès cela rend le code difficile à suivre et à corriger, car une fonction peut modifier une variable à distance sans que ce soit visible. La bonne pratique préférée par les développeurs est de passer les valeurs en **paramètres** et de récupérer le résultat avec **return**.

4. Le tableau \$GLOBALS et les variables statiques

Le tableau \$GLOBALS : une autre façon d'accéder aux variables globales

PHP met à disposition un tableau spécial nommé `$GLOBALS` qui contient toutes les variables globales du script. On peut l'utiliser à l'intérieur d'une fonction sans avoir besoin du mot-clé `global`.

```
<?php
$ville = "Dakar";
$pays  = "Sénégal";

function afficherLocalisation() {
    // Accès aux variables globales via $GLOBALS
    echo $GLOBALS["ville"] . ", " . $GLOBALS["pays"];
}

afficherLocalisation(); // Affiche : Dakar, Sénégal
?>
```

DIFFÉRENCE ENTRE GLOBAL ET \$GLOBALS

Les deux permettent d'accéder aux variables globales depuis une fonction. La différence : `global $x` crée un lien direct avec la variable globale, on peut ensuite utiliser `$x` normalement. `$GLOBALS["x"]` accède directement au tableau sans déclaration préalable, utile quand le nom de la variable est dynamique.

La variable statique : mémoriser une valeur entre les appels

DÉFINITION

Une variable **statique** est déclarée avec le mot-clé `static` à l'intérieur d'une fonction. Contrairement à une variable locale ordinaire qui est détruite à chaque fin d'appel, une variable statique **conserve sa valeur** d'un appel à l'autre de la fonction.

```
<?php
function compter() {
    static $nb = 0; // initialisée une seule fois (au premier appel)
    $nb++;
    echo "Appel numéro : " . $nb . "<br>";
}

compter(); // Affiche : Appel numéro : 1
compter(); // Affiche : Appel numéro : 2
compter(); // Affiche : Appel numéro : 3
?>
```

ANALOGIE SIMPLE

Une variable statique, c'est comme un post-it collé sur la porte de la salle de classe. À chaque fois que quelqu'un entre (appel de la fonction), il regarde le post-it, le met à jour, et le laisse en place pour le prochain. Une variable locale ordinaire, c'est un post-it qu'on jette à chaque sortie.

5. Comparaison et bonnes pratiques

Tableau comparatif des trois portées

Type	Déclaration	Accessible où ?	Durée de vie
Locale	Dans une fonction, sans mot-clé	Dans la fonction uniquement	Détruite à la fin de la fonction
Globale	En dehors des fonctions	Hors fonctions (ou avec <code>global</code>)	Pendant toute la durée du script
Statique	Dans une fonction, avec <code>static</code>	Dans la fonction uniquement	Conservée entre les appels de la fonction

La bonne pratique : paramètres et return plutôt que global

Utiliser `global` dans une fonction crée une dépendance cachée : la fonction dépend d'une variable externe sans que cela soit visible dans sa signature. Cela rend le code difficile à tester et à relire. La solution recommandée est de passer les informations en paramètre et de renvoyer le résultat avec `return` .

```
<?php
// ✖ Mauvaise pratique : utilisation de global
$tva = 0.2;
function calculerPrixTTC_mauvais($prixHT) {
    global $tva;
    return $prixHT * (1 + $tva);
}

// ✔ Bonne pratique : tout est passé en paramètre
function calculerPrixTTC($prixHT, $tva) {
    return $prixHT * (1 + $tva);
}

echo calculerPrixTTC(100, 0.2); // Affiche : 120
?>
```

RÈGLE D'OR À RETENIR

Une bonne fonction doit être **autonome** : toutes les informations dont elle a besoin lui sont transmises en paramètres, et elle renvoie son résultat avec `return` . Elle ne doit pas dépendre de variables externes cachées. C'est ainsi que les développeurs professionnels écrivent leur code.

6. Résumé complet sur la portée des variables

Concept	Exemple de code	Résultat
Variable locale	<code>function f() { \$x = 1; }</code>	\$x n'existe que dans f()
Variable globale	<code>\$x = 1;</code> (hors fonction)	\$x visible partout hors fonction
Accès global depuis fonction	<code>global \$x;</code>	\$x globale utilisable dans la fonction
Via \$GLOBALS	<code>\$GLOBALS["x"]</code>	Accès direct à la variable globale \$x
Variable statique	<code>static \$nb = 0;</code>	\$nb garde sa valeur entre les appels

Les questions à se poser pour identifier la portée

- Est-ce que la variable est créée dans une fonction ?** → Si oui, elle est locale et n'existe que là.
- Est-ce que la variable est créée en dehors de toute fonction ?** → Si oui, elle est globale, mais il faut `global` pour y accéder depuis une fonction.
- A-t-on besoin de mémoriser sa valeur entre plusieurs appels d'une fonction ?** → Utiliser `static`.
- A-t-on besoin de transmettre une valeur à une fonction ?** → Utiliser un paramètre, c'est toujours la meilleure solution.

🔔 La phrase à retenir pour ta présentation

"En PHP, chaque variable a une portée : une variable locale vit uniquement dans la fonction où elle est créée et disparaît quand la fonction se termine. Une variable globale est créée hors des fonctions mais pour l'utiliser à l'intérieur d'une fonction, il faut écrire le mot-clé 'global'. Enfin, une variable statique est locale mais garde sa valeur en mémoire d'un appel à l'autre de la fonction."

Erreurs courantes à éviter

- Croire qu'une variable créée dans une fonction est accessible en dehors — ce n'est jamais le cas.
- Croire qu'une variable globale est automatiquement visible dans une fonction — il faut toujours écrire `global`.
- Confondre `static` (variable qui garde sa valeur entre les appels) et `global` (variable partagée avec l'extérieur).
- Abuser de `global` dans un gros projet : cela rend le code très difficile à comprendre et à maintenir.
- Oublier que deux fonctions ayant une variable avec le même nom (`$total` par exemple) n'ont aucun lien l'une avec l'autre.